

DSBDA Practical Journal

Data Science & Big Data Analytics

Complete Viva-Ready Explanation of All Practicals

Assignments A1 - A10 | B1, B2, B4 (Hadoop & Spark)

A1 - Data Wrangling on Titanic Dataset

Topic	Data Wrangling & Pre-processing
Dataset	train.csv (Titanic passenger data)
Library	pandas, numpy, matplotlib, seaborn, sklearn

What is this Practical About?

This practical focuses on cleaning and preparing raw data for analysis. Real-world datasets are messy — they have missing values, wrong data types, and unscaled numbers. Data wrangling means fixing all these issues so the data is ready for machine learning or analysis.

Step-by-Step Logic

Step 1: Load & Explore Data

The Titanic CSV file is loaded using `pd.read_csv()`. Initial exploration is done using:

- `df.head()` – first 5 rows
- `df.shape` – number of rows and columns
- `df.info()` – column names, data types, non-null counts
- `df.describe()` – statistical summary (mean, std, min, max)
- `df.isnull().sum()` – count of missing values per column

Step 2: Handle Missing Values

- Age column: filled with mean age (`df['Age'].fillna(df['Age'].mean())`)
- Embarked column: filled with mode (most frequent value)
- Cabin column: dropped entirely because too many values are missing (using `df.drop()`)

Step 3: Data Type Conversion

- Sex and Embarked columns converted to category type using `astype('category')`
- This saves memory and makes processing faster

Step 4: Encoding Categorical Variables

- Sex column: label encoded – male=0, female=1 using `df['Sex'].map()`
- Embarked column: One-Hot Encoded using `pd.get_dummies()` — creates separate binary columns for each category (Embarked_C, Embarked_Q, Embarked_S)

Step 5: Normalization

- Age and Fare columns normalized using `MinMaxScaler` from `sklearn`
- `MinMaxScaler` transforms values to range [0, 1]
- Formula: $(x - \min) / (\max - \min)$

Step 6: Save Cleaned Data

- Final cleaned dataset saved to `cleaned_titanic.csv` using `df.to_csv()`

Key Viva Questions

Q: Why do we handle missing values?

A: Missing values cause errors in ML models. We fill them with mean/mode to preserve data rather than deleting rows.

Q: Why use MinMaxScaler?

A: Features with large values (like Fare) would dominate ML algorithms. Normalization brings all features to the same scale.

Q: What is One-Hot Encoding?

A: It converts a categorical column with N categories into N binary (0/1) columns, so ML models can process them numerically.

Q: Difference between Label Encoding and One-Hot Encoding?

A: Label Encoding assigns numbers (0,1,2...) to categories; it implies an order. One-Hot Encoding creates binary columns with no implied order. Use One-Hot for nominal data.

A2 - Handling Outliers on Student Performance Dataset

Topic	Descriptive Statistics, Outlier Detection & Treatment
Dataset	Manually created student dataset (10 students)
Library	pandas, numpy, matplotlib, seaborn, scipy

What is this Practical About?

This practical deals with detecting and handling outliers — values that are abnormally high or low compared to the rest. A student with Maths marks of 300 is clearly an error. The practical also checks for inconsistencies (like negative attendance) and applies log transformation to reduce skewness.

Step-by-Step Logic

Step 1: Create & Explore Dataset

A student dataset is manually created with deliberate errors: a Maths mark of 300 (clearly out of range) and negative attendance. The dataset is explored using `info()`, `shape`, `isnull().sum()`.

Step 2: Handle Missing Values

- Missing values in Maths, Science, English filled using their mean values
- `df['Maths'].fillna(df['Maths'].mean())` ensures no NaN values remain

Step 3: Check Inconsistencies

- Attendance column converted to float type
- Negative attendance values are detected and replaced with the mean of valid attendance values

Step 4: Visualize with Boxplot

- `sns.boxplot()` draws a box showing Q1, median, Q3, and whiskers
- Points outside the whiskers are outliers

Step 5: IQR Method for Outlier Detection

The IQR (Interquartile Range) method is a standard way to detect outliers:

```
Q1 = df['Maths'].quantile(0.25)
Q3 = df['Maths'].quantile(0.75)
IQR = Q3 - Q1
Lower Limit = Q1 - 1.5 * IQR
Upper Limit = Q3 + 1.5 * IQR
```

- Any value outside these limits is an outlier
- Outliers are replaced with the median value to reduce their impact

Step 6: Log Transformation

- `np.log()` applied to Maths column
- Reduces positive skewness — makes data distribution more symmetric (bell-shaped)
- Skewness before and after transformation is compared using `df.skew()`

Key Viva Questions

Q: What is an outlier?

A: A data point that is significantly different from other observations. It can be due to data entry errors or genuine rare events.

Q: What is IQR?

A: Interquartile Range = $Q3 - Q1$. It measures the spread of the middle 50% of data. Values outside $Q1 - 1.5 \cdot IQR$ and $Q3 + 1.5 \cdot IQR$ are outliers.

Q: Why replace outliers with median instead of mean?

A: Mean is affected by outliers itself, so replacing with mean doesn't fully solve the problem. Median is more robust.

Q: What is skewness?

A: Skewness measures the asymmetry of a distribution. Positive skew means a long right tail. Log transformation reduces positive skewness.

A3 - Descriptive Statistics: Adult & Iris Datasets

Topic	Statistical Analysis using GroupBy
Dataset	adult.csv (census data) and Iris.csv
Library	pandas, numpy

What is this Practical About?

This practical has two parts. Part 1 uses the Adult dataset to group people by age and compute working hours statistics. Part 2 uses the Iris dataset to group by species and calculate descriptive statistics for each species. The goal is to understand how groupBy operations summarize data.

Part 1 – Adult Dataset

Step 1: Load & Rename Columns

The adult.csv dataset (census data) is loaded. Columns are renamed for clarity (Age, Workclass, Education, Income, etc.).

Step 2: Create Age Groups using pd.cut()

```
df['Age_Group'] = pd.cut(df['Age'], bins=[0,25,45,65,100],  
labels=['Young', 'Adult', 'Middle_Age', 'Senior'])
```

- pd.cut() divides continuous Age values into labeled bins
- Creates a new categorical column Age_Group

Step 3: GroupBy and Aggregate

- df.groupby('Age_Group')['Hours_per_week'].agg(['mean','median','min','max','std'])
- Groups all rows by Age_Group, then calculates statistics on Hours_per_week
- .apply(list) collects all individual values as a list per group

Part 2 – Iris Dataset

Step 1: Group by Species

- iris.groupby('Species') creates 3 groups: Iris-setosa, Iris-versicolor, Iris-virginica

Step 2: Loop and Print Stats

- Loops through each species group
- Prints mean, median, min, max, standard deviation for sepal and petal measurements
- .mean(numeric_only=True) avoids errors on non-numeric columns

Key Viva Questions

Q: What does groupby() do?

A: It splits the DataFrame into groups based on a column, applies a function to each group, and combines the results.

Q: What is pd.cut()?

A: It bins continuous numerical data into discrete intervals/categories. Similar to creating age groups like Young, Adult, Senior.

Q: What is standard deviation?

A: It measures the spread of data around the mean. A high std means data is widely spread; low std means data is close to the mean.

Q: Difference between mean and median?

A: Mean is average (sum/count), sensitive to outliers. Median is the middle value, more robust to outliers.

A4 - Linear Regression on Boston Housing Dataset

Topic	Supervised ML – Linear Regression
Dataset	BostonHousing.csv
Library	pandas, numpy, seaborn, matplotlib, sklearn

What is this Practical About?

Linear Regression is used to predict house prices (medv = median value in \$1000s) based on features like crime rate, number of rooms, distance to employment centers, etc. This is a supervised learning regression problem — we train a model on known data and use it to predict unknown values.

Step-by-Step Logic

Step 1: Load & Explore Data

- BostonHousing.csv has 506 rows and 14 features including crim, rm (rooms), lstat, medv
- `df.describe()` gives statistical summary
- `df.corr()` shows correlation between all features

Step 2: Correlation Heatmap

- `sns.heatmap(df.corr())` visualizes correlations
- Values close to +1 or -1 show strong correlation
- Helps identify which features most influence house price

Step 3: Train-Test Split

```
X = df.drop('medv', axis=1)    # All features
y = df['medv']                 # Target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

- 80% data for training, 20% for testing
- `random_state=42` ensures reproducibility

Step 4: Train Linear Regression Model

```
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
```

- `fit()` trains the model by finding best coefficients
- `predict()` applies the learned formula to test data

Step 5: Evaluate Model

- MSE (Mean Squared Error): average squared difference between actual and predicted values
- R^2 Score: how well the model explains variance. $R^2=1$ is perfect; $R^2=0$ means the model is no better than using the mean

Step 6: Scatter Plot

- Actual vs Predicted prices plotted
- Red diagonal line = perfect predictions
- Points close to the diagonal = good model performance

Key Viva Questions

Q: What is Linear Regression?

A: A supervised ML algorithm that models the relationship between a dependent variable (y) and one or more independent variables (X) using a straight line: $y = mx + c$.

Q: What is MSE?

A: Mean Squared Error = average of $(\text{actual} - \text{predicted})^2$. Lower MSE means better model. Squaring penalizes larger errors more.

Q: What is R² score?

A: Coefficient of Determination. Ranges from 0 to 1. Indicates what proportion of variance in y is explained by X. $R^2=0.7$ means model explains 70% of variance.

Q: What is the difference between training and test data?

A: Training data is used to build/train the model. Test data is unseen data used to evaluate how well the model generalizes.

A5 - Logistic Regression on Social Network Ads

Topic	Supervised ML – Logistic Regression (Classification)
Dataset	Social_Network_Ads.csv
Library	pandas, sklearn

What is this Practical About?

Logistic Regression is a classification algorithm used to predict binary outcomes. Here, we predict whether a user will Purchase (1) or Not Purchase (0) a product based on their Age and Estimated Salary. The practical also evaluates the model using Confusion Matrix, Accuracy, Precision, and Recall.

Step-by-Step Logic

Step 1: Load & Prepare Data

- Dataset has Gender, Age, EstimatedSalary, Purchased columns
- Gender encoded: Male=1, Female=0

Step 2: Feature Selection

```
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

- Using only Age and Salary as features to predict Purchased

Step 3: Train-Test Split

- `test_size=0.25` means 75% train, 25% test

Step 4: Feature Scaling

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

- StandardScaler: converts data to mean=0, std=1
- Logistic Regression is sensitive to feature scales — scaling is important
- `fit_transform` on train, only `transform` on test (to prevent data leakage)

Step 5: Train & Predict

- `LogisticRegression().fit()` trains the model
- `model.predict()` generates class labels (0 or 1)

Step 6: Confusion Matrix & Metrics

- Confusion Matrix: 2x2 table showing TP, TN, FP, FN
- Accuracy = $(TP + TN) / \text{Total}$
- Precision = $TP / (TP + FP)$ — of all predicted positives, how many are correct?
- Recall = $TP / (TP + FN)$ — of all actual positives, how many were caught?
- Error Rate = $1 - \text{Accuracy}$

Key Viva Questions

Q: Difference between Linear and Logistic Regression?

A: Linear Regression predicts continuous values. Logistic Regression predicts probabilities (0 to 1) and classifies into categories. It uses the sigmoid function.

Q: What is the Confusion Matrix?

A: A 2x2 matrix showing: TP (correctly predicted positive), TN (correctly predicted negative), FP (predicted positive but actually negative), FN (predicted negative but actually positive).

Q: When is Precision more important than Recall?

A: Precision matters when false positives are costly (e.g., spam detection). Recall matters when false negatives are costly (e.g., disease detection).

Q: Why use StandardScaler?

A: Logistic Regression uses gradient descent which is sensitive to feature scale. StandardScaler normalizes features to mean 0, std 1.

A6 - Naive Bayes Classifier on Iris Dataset

Topic	Supervised ML – Naive Bayes Classification
Dataset	iris.csv
Library	pandas, numpy, sklearn

What is this Practical About?

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It's called 'Naive' because it assumes all features are independent of each other. Here, it classifies Iris flowers into 3 species (setosa, versicolor, virginica) based on sepal and petal measurements. GaussianNB is used since the features are continuous.

Step-by-Step Logic

Step 1: Load Data

- iris.csv has 150 rows: SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species

Step 2: Split Features & Target

```
X = data.iloc[:, :-1] # All columns except last (features)
y = data.iloc[:, -1]  # Last column (Species - target)
```

Step 3: Train-Test Split

- test_size=0.3 (30% test, 70% train)

Step 4: Train Gaussian Naive Bayes

```
model = GaussianNB()
model.fit(X_train, y_train)
```

- GaussianNB assumes features follow a Gaussian (normal) distribution

Step 5: Predict & Evaluate

- model.predict() returns predicted species for test data
- Confusion Matrix computed — for 3-class problem, it's a 3x3 matrix
- For each class: TP, FP, FN, TN calculated using the one-vs-rest approach
- Accuracy = (total correct predictions) / (total predictions)
- Error Rate = 1 - Accuracy

Key Viva Questions

Q: What is Bayes' Theorem?

A: $P(A|B) = P(B|A) * P(A) / P(B)$. It calculates the probability of A given B. In classification, it finds the most probable class given the features.

Q: Why is it called Naive?

A: Because it assumes all features are independent, which is rarely true in practice. Despite this 'naive' assumption, it works surprisingly well.

Q: What is GaussianNB?

A: A variant of Naive Bayes that assumes each feature follows a Gaussian (normal/bell curve) distribution. Used for continuous data.

Q: What is a multi-class Confusion Matrix?

A: For N classes, it's an $N \times N$ matrix. Each cell (i,j) shows how many instances of class i were predicted as class j .

A7 - Text Analytics using NLTK

Topic	Natural Language Processing (NLP) – Text Preprocessing & TF-IDF
Dataset	Custom paragraph about text analytics
Library	nltk, sklearn (TfidfVectorizer)

What is this Practical About?

Text data cannot be directly fed into ML models. This practical covers the standard NLP preprocessing pipeline: Tokenization, POS Tagging, Stopword Removal, Stemming, Lemmatization, and finally TF-IDF vectorization to convert text into numbers.

Step-by-Step Logic

Step 1: Download NLTK Resources

- `nltk.download()` downloads punkt (tokenizer), stopwords, wordnet (for lemmatizer), pos_tagger

Step 2: Tokenization

```
tokens = word_tokenize(document)
```

- Splits the paragraph into individual words and punctuation marks
- 'Text analytics is important' → ['Text', 'analytics', 'is', 'important']

Step 3: POS Tagging

```
pos_tags = nltk.pos_tag(tokens)
```

- Labels each word with its Part of Speech: NN (noun), VB (verb), JJ (adjective), etc.
- Output: [('Text', 'NN'), ('analytics', 'NNS'), ('is', 'VBZ')]

Step 4: Stopword Removal

```
filtered_tokens = [w for w in tokens if w.lower() not in stop_words]
```

- Removes common words like 'is', 'the', 'and', 'a' that don't add meaning
- `stopwords.words('english')` contains 179 English stopwords

Step 5: Stemming

```
stemmer = PorterStemmer()
stemmed = [stemmer.stem(w) for w in filtered_tokens]
```

- Cuts words to their root form: 'running' → 'run', 'techniques' → 'techniqu'
- Fast but can produce non-real words

Step 6: Lemmatization

```
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(w) for w in filtered_tokens]
```

- Converts words to their dictionary base form: 'running' → 'run', 'better' → 'good'
- Slower but produces actual words; uses WordNet dictionary

Step 7: TF-IDF Vectorization

- `TfidfVectorizer` converts a corpus of 3 documents into a numeric matrix
- TF (Term Frequency): how often a word appears in a document

- IDF (Inverse Document Frequency): penalizes words that appear in many documents (common words)
- $TF\text{-}IDF = TF * IDF$ — gives high score to words that are frequent in one document but rare overall
- Output is a matrix: rows = documents, columns = unique words

Key Viva Questions

Q: Difference between Stemming and Lemmatization?

A: Stemming chops word endings (fast, crude, may produce non-words). Lemmatization uses a dictionary to find the correct base form (slower, more accurate, produces real words).

Q: What are stopwords?

A: Frequently used words in a language (is, the, and, a) that carry little meaning and are removed before NLP processing.

Q: What is TF-IDF?

A: Term Frequency–Inverse Document Frequency. It weighs words by how important they are to a document. High TF-IDF = word is frequent in this doc but rare in others.

Q: What is POS tagging?

A: Part-of-Speech tagging assigns grammatical tags (noun, verb, adjective) to each word in a sentence.

A8 - Data Visualization on Titanic Dataset

Topic	Descriptive Statistics & Visualization (Histograms)
Dataset	titanic.csv
Library	pandas, seaborn, matplotlib

What is this Practical About?

This practical focuses on visualizing data distributions using Histograms. A histogram shows how data is spread across intervals. Here, the distribution of ticket fares is visualized after handling missing values in the Titanic dataset.

Step-by-Step Logic

Step 1: Load & Inspect

- titanic.csv loaded using `pd.read_csv()`
- `df.info()` shows column types and null counts
- `df.isnull().sum()` identifies missing data

Step 2: Clean Data

- age: missing values filled with median (more robust than mean for skewed data)
- embarked: filled with mode
- fare: filled with median
- deck/cabin column dropped (too many missing values)
- `df.drop_duplicates()` removes duplicate rows

Step 3: Plot Histogram of Fare

```
sns.histplot(df['fare'], bins=30, kde=True)
```

- `bins=30` divides the fare range into 30 intervals
- `kde=True` adds a smooth curve (Kernel Density Estimate) showing the overall distribution shape
- The fare distribution is right-skewed — most passengers paid low fares, few paid very high fares

Key Viva Questions

Q: What is a Histogram?

A: A bar chart showing the frequency distribution of a continuous variable. The x-axis shows value ranges (bins) and y-axis shows count of values in each bin.

Q: What is KDE?

A: Kernel Density Estimate — a smooth curve that represents the probability distribution of a variable. It's like a smooth version of the histogram.

Q: What does right-skewed mean?

A: The distribution has a long tail to the right. Most values are low, but a few are very high. $\text{Mean} > \text{Median}$ in a right-skewed distribution.

Q: Why fill missing values with median instead of mean for fare?

A: Fare is likely right-skewed. The median is not affected by extreme high values, making it a better representative central value.

A9 - Box Plot Visualization on Titanic Dataset

Topic	Data Visualization – Box Plot Analysis
Dataset	titanic.csv
Library	pandas, seaborn, matplotlib

What is this Practical About?

This practical uses Box Plots to analyze age distribution by gender and survival status. Box plots summarize data distribution in 5 numbers: minimum, Q1, median, Q3, and maximum. They also visually show outliers.

Step-by-Step Logic

Step 1: Load & Clean

- titanic.csv loaded
- Rows with missing age values dropped using `dropna(subset=['age'])`

Step 2: Box Plot — Age by Gender & Survival

```
sns.boxplot(x='sex', y='age', hue='survived', data=titanic)
```

- `x='sex'`: groups by Male/Female on x-axis
- `y='age'`: shows age distribution on y-axis
- `hue='survived'`: further splits each group by survived (0=No, 1=Yes)
- 4 boxes total: Male-Died, Male-Survived, Female-Died, Female-Survived

Step 3: Observations & Inferences

- Higher proportion of females survived (Women and children first policy)
- Surviving females were slightly younger on average
- Surviving males tended to be younger than non-surviving males
- Children had better survival rates
- Outliers visible as individual points beyond the whiskers

Understanding a Box Plot

- Box spans from Q1 (25th percentile) to Q3 (75th percentile) — the middle 50% of data
- Line inside box = Median (Q2, 50th percentile)
- Whiskers extend to $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$
- Points beyond whiskers = Outliers

Key Viva Questions

Q: What is a Box Plot?

A: A graphical representation of data distribution using 5-number summary: minimum, Q1, median, Q3, maximum. Also called Box-and-Whisker plot.

Q: What does 'hue' parameter do in seaborn?

A: It adds a third dimension by color-coding the plot based on another categorical variable (here, survived vs not survived).

Q: What information does a box plot provide?

A: Central tendency (median), spread (IQR), symmetry (box position), and outliers (points beyond whiskers).

Q: Why use Box Plot over Histogram?

A: Box plots are better for comparing distributions across multiple categories side by side. Histograms are better for showing the shape of a single distribution.

A10 - Data Visualization on Iris Dataset (Histograms & Boxplots)

Topic	Feature Type Identification, Histograms, Boxplots, Outlier Detection
Dataset	iris.csv
Library	pandas, matplotlib, seaborn

What is this Practical About?

This practical covers comprehensive visualization of the Iris dataset — identifying feature types, plotting histograms for all numeric features, drawing boxplots, and detecting outliers using the IQR method. It's a complete EDA (Exploratory Data Analysis) exercise.

Step-by-Step Logic

Step 1: Load & Identify Feature Types

- For each column: if dtype == 'object' → Nominal/Categorical, else → Numeric
- Result: SepalLength, SepalWidth, PetalLength, PetalWidth are Numeric; Species is Nominal

Step 2: Histograms for All Numeric Features

```
df[numeric_cols].hist(figsize=(10,8), bins=20)
```

- Plots histograms for all 4 numeric features in one figure
- Shows whether each feature is normally distributed, skewed, or bimodal

Step 3: Boxplots for All Numeric Features

```
fig, axes = plt.subplots(2, 2, figsize=(10,8))
```

- Creates a 2x2 grid of subplots
- `sns.boxplot()` drawn for each feature individually
- Helps compare spread and identify outliers for each measurement

Step 4: IQR-based Outlier Detection

```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df[col] < Q1-1.5*IQR) | (df[col] > Q3+1.5*IQR)]
```

- Applied to all 4 numeric features
- Reports the count of outliers for each feature

Step 5: Statistical Summary

- `df.describe()` provides count, mean, std, min, 25%, 50%, 75%, max for each feature

Key Viva Questions

Q: What are the types of data in the Iris dataset?

A: Numeric (continuous): SepalLength, SepalWidth, PetalLength, PetalWidth. Nominal (categorical): Species (setosa, versicolor, virginica).

Q: What is EDA?

A: Exploratory Data Analysis — the process of analyzing data using visual and statistical methods to understand patterns, relationships, and anomalies before modeling.

Q: How many outliers does Iris typically have?

A: SepalWidth has a few outliers. PetalLength and PetalWidth have very few. The dataset is generally clean.

Q: What does `plt.subplots(2,2)` do?

A: Creates a figure with 2 rows and 2 columns of subplot axes, allowing 4 plots to be displayed in one figure.

B1 - Hadoop MapReduce: Word Count

Topic	Big Data – Hadoop MapReduce
Language	Java
Program	WordCount.java

What is this Practical About?

Word Count is the 'Hello World' of Hadoop. It counts how many times each word appears in a text file. The program demonstrates the MapReduce paradigm — distributing the counting task across many nodes in a cluster. Even on a single machine, it shows the map-shuffle-reduce flow.

MapReduce Concept

- MapReduce is a programming model for processing large datasets in parallel across a cluster
- Map Phase: processes input and produces key-value pairs
- Shuffle & Sort Phase: groups all values with the same key together (done automatically by Hadoop)
- Reduce Phase: aggregates values for each key

Code Explanation

Mapper Class (TokenizerMapper)

```
extends Mapper<Object, Text, Text, IntWritable>
```

- Input: each line of the text file
- StringTokenizer splits each line into words
- For each word, emits (word, 1) using context.write(word, one)
- Example: 'Hello World' → (Hello, 1), (World, 1)

Reducer Class (IntSumReducer)

```
extends Reducer<Text, IntWritable, Text, IntWritable>
```

- Receives all values for each key: e.g., (Hello, [1, 1, 1])
- Sums all values: Hello → 3
- Emits (Hello, 3) as final output

Driver Class (main method)

- Configures the Hadoop Job object
- Sets Mapper, Reducer, input/output key-value types
- Sets input path (HDFS or local) and output path
- job.waitForCompletion(true) runs the job

How to Run

```
mv B1.java WordCount.java
javac -classpath $(hadoop classpath) -d . WordCount.java
jar cf wc.jar WordCount*.class
hadoop jar wc.jar WordCount input output
cat output/part-r-00000
```

Key Viva Questions

Q: What is MapReduce?

A: A programming model that splits a big task into Map (process and emit key-value pairs) and Reduce (aggregate values per key) phases, running in parallel across a cluster.

Q: What does the Mapper output in Word Count?

A: For each word in the input, it outputs (word, 1). If 'hello' appears 3 times, 3 separate (hello, 1) pairs are emitted.

Q: What happens in the Shuffle phase?

A: Hadoop automatically groups all values with the same key together. So (hello,1),(hello,1),(hello,1) becomes (hello, [1,1,1]) before being sent to the Reducer.

Q: What is IntWritable?

A: A Hadoop-specific serializable class for integers, used instead of Java's int for network transmission across nodes.

Q: What is HDFS?

A: Hadoop Distributed File System — stores large files across multiple machines in chunks (128MB default), with replication for fault tolerance.

B2 - Hadoop MapReduce: Log File Analysis

Topic	Big Data – Hadoop MapReduce with 3 Files
Language	Java (3 separate files)
Files	LogMapper.java, LogReducer.java, LogDriver.java

What is this Practical About?

This practical counts the frequency of each log level (INFO, WARN, ERROR, DEBUG) in a system log file using Hadoop MapReduce. It's split into 3 separate Java files — Mapper, Reducer, Driver — to demonstrate professional code organization. The log format is: date time LOGLEVEL message.

Input Log Format

```
2024-01-15 10:00:01 INFO  Server started successfully
2024-01-15 10:02:10 WARN  High memory usage detected
2024-01-15 10:03:15 ERROR Disk read failure on /dev/sda
```

Code Explanation

LogMapper.java

- Reads each line of the log file
- Splits line by space: `parts = line.split(' ')`
- `parts[2]` is the log level (3rd element: 0=date, 1=time, 2=LOGLEVEL)
- Emits (LOGLEVEL, 1) → e.g., (ERROR, 1), (INFO, 1)

LogReducer.java

- Receives all 1s for each log level: e.g., (ERROR, [1,1,1,1])
- Sums them up and outputs (ERROR, 4)

LogDriver.java

- Sets up the Hadoop Job
- Registers LogMapper and LogReducer classes
- Sets input/output key types and file paths
- Entry point for the entire MapReduce job

How to Run

```
javac -classpath $(hadoop classpath) -d . LogDriver.java LogMapper.java
LogReducer.java
jar cf log.jar LogDriver.class LogMapper.class LogReducer.class
hadoop jar log.jar LogDriver input output
cat output/part-r-00000
```

Expected output:

```
DEBUG 2
ERROR 4
INFO 6
WARN 3
```

Key Viva Questions

Q: How does LogMapper extract the log level?

A: It splits each log line by spaces and reads index 2 (the third word), which is the log level in the format: date time LOGLEVEL message.

Q: Why are there 3 separate files in B2 but only 1 in B1?

A: B2 follows a more professional/modular structure. B1 uses inner classes within one file. Both approaches work — separate files improve readability and reusability.

Q: What happens if the output directory already exists?

A: Hadoop throws an error. The output directory must not exist before running the job. Delete it with:
`hadoop fs -rm -r output`

Q: What is the role of the Driver class?

A: The Driver (main class) configures and submits the Hadoop job. It tells Hadoop which Mapper and Reducer to use, what types the keys/values are, and where the input/output files are.

B4 - Apache Spark: Prime Numbers using Scala

Topic	Big Data – Apache Spark with Scala
Language	Scala
Program	prime.scala (Object: Prime)

What is this Practical About?

Apache Spark is a fast, in-memory distributed data processing framework. This practical uses Spark's RDD (Resilient Distributed Dataset) to find all prime numbers from 1 to 50. It demonstrates how Spark creates distributed datasets and applies transformations (filter) and actions (collect).

What is Spark vs Hadoop MapReduce?

- Hadoop MapReduce writes intermediate results to disk (slow)
- Spark processes data in-memory (up to 100x faster)
- Spark has a richer API: map, filter, reduce, collect, count, etc.
- Spark supports multiple languages: Scala, Python (PySpark), Java, R
- Spark is better for iterative algorithms (ML) and real-time streaming

Code Explanation

isPrime Function

```
def isPrime(n: Int): Boolean = {  
  if (n <= 1) false  
  else !(2 until n).exists(n % _ == 0)  
}
```

- Returns false if $n \leq 1$ (0, 1 are not prime)
- $(2 \text{ until } n).exists(n \% _ == 0)$ — checks if any number from 2 to $n-1$ divides n
- If any such divisor exists → not prime
- The $!$ negates it: prime = no divisor found

Main Method

```
val conf = new SparkConf().setAppName('Prime').setMaster('local')  
val sc = new SparkContext(conf)
```

- SparkConf: configuration for the Spark application
- setMaster('local'): run on local machine (no cluster needed)
- SparkContext (sc): the entry point for Spark operations

RDD Operations

```
val numbers = sc.parallelize(1 to 50)  
val primes = numbers.filter(isPrime)  
primes.collect().foreach(println)
```

- `sc.parallelize()` creates an RDD from a Scala range (1 to 50)
- `.filter(isPrime)` applies the `isPrime` function to each element — keeps only primes
- `.collect()` brings all distributed results back to the driver program
- `.foreach(println)` prints each prime number
- `sc.stop()` — releases Spark resources

Key Viva Questions

Q: What is an RDD?

A: Resilient Distributed Dataset — the fundamental data structure in Spark. It's an immutable, distributed collection of objects that can be processed in parallel across a cluster.

Q: What is the difference between transformation and action in Spark?

A: Transformations (map, filter) are lazy — they don't execute immediately but define a computation graph. Actions (collect, count, save) trigger the actual execution.

Q: What does `sc.parallelize()` do?

A: It distributes a local collection (array, range) across the Spark cluster as an RDD, enabling parallel processing.

Q: What is `setMaster('local')` in `SparkConf`?

A: It tells Spark to run on the local machine using a single thread. 'local[*]' uses all available CPU cores. For a real cluster, you'd provide the master URL.

Q: Why is Spark faster than Hadoop?

A: Spark processes data in-memory (RAM) instead of writing to disk between stages. This makes iterative computations (like ML) much faster.

Quick Reference: All Practicals at a Glance

No.	Topic	Dataset	Algorithm/Tool	Key Output
A1	Data Wrangling	Titanic (train.csv)	MinMaxScaler, Encoding	Cleaned CSV
A2	Outlier Handling	Student Marks	IQR, Log Transform	Cleaned dataset
A3	Descriptive Stats	Adult + Iris	GroupBy, pd.cut()	Group-wise stats
A4	Linear Regression	Boston Housing	LinearRegression	MSE, R ² score
A5	Logistic Regression	Social Network Ads	LogisticRegression	Confusion Matrix
A6	Naive Bayes	Iris	GaussianNB	Accuracy, TP/FP/TN/FN
A7	Text Analytics	Custom text	NLTK, TF-IDF	Vectorized matrix
A8	Visualization	Titanic	Histogram + KDE	Fare distribution
A9	Box Plot	Titanic	Boxplot (seaborn)	Survival by gender/age
A10	EDA + Visualization	Iris	Hist, Boxplot, IQR	Outlier counts
B1	Word Count	Text file	Hadoop MapReduce	Word frequencies
B2	Log Analysis	Log file	Hadoop MapReduce	Log level counts
B4	Prime Numbers	Range 1-50	Apache Spark + Scala	Prime numbers

All the best for your practical tomorrow!

Study the Key Viva Questions for each practical — they cover the most likely examiner questions.